Installing Indi from GitHub

Oct 8, 2026 · @Steve

Overview

A new installation is a separate copy of Indi with its own Linux user, Gmail mailbox, Anthropic key, settings, data and timers. It shares nothing with the live Indi except the code it pulls from GitHub. Expect about an hour the first time, most of it on the Google sign-in steps.

These steps cover two cases:

- **A second installation on the existing Hetzner server (77.42.46.44).** This is the simplest test and the one the examples use, with the Linux user `indi2`.
- **A fresh server.** Same steps, plus a short preparation section in "Before you start".

When you finish, the new installation will:

- check its own Gmail inbox on a systemd timer and reply under its own settings;
- keep its people, ledger, approvals and terms acceptances in its own `data/` folder;
- be able to `git pull` code updates from GitHub, but not push to it.

Commands are written for a bash session on the server. Lines that start with `sudo` or that say "as root" need root; everything else runs as the new user. Wherever you see `indi2`, substitute your own choice of name.

The repository also holds `README.md` and `README-hetzner.md` from the original setup. Where this guide says "check the README", those are the files meant.

Before you start

Have these ready:

Item	What it is for	Where it comes from
Root SSH access to the server	Creating the user and installing services	Hetzner (you already have it for 77.42.46.44)

Item	What it is for	Where it comes from
Access to the GitHub account `indiartificial`	Adding a deploy key to `indiartificial/indi-harness`	github.com, signed in with Google
A Gmail address for the new installation	The mailbox Indi reads and sends from	A new Google account, or a spare one
Access to the Google Cloud project that holds Indi's OAuth client	Adding the new Gmail address as a test user	console.cloud.google.com
An Anthropic API key	Indi's model calls	console.anthropic.com, ideally a separate key per installation
The owner's email address	Where operator mail, approvals and alerts go	You decide
A Windows PowerShell window	The SSH tunnel used by the Google sign-in	Your PC
`requirements.txt` committed to the repository	Installing the right Python packages	Done earlier; confirm with `git ls-files requirements.txt` on the live install

Extra steps for a fresh server

Skip this on 77.42.46.44, which already has them. On a new Ubuntu or Debian server, as root:

```
apt update && apt -y upgrade
apt -y install python3 python3-venv python3-pip git
timedatectl set-timezone America/New_York   # optional; Indi has its own
INDI_TIMEZONE setting
```

Check the Python version with `python3 --version`. It should be the same minor version as the live server, so the pinned packages install cleanly. Run `/home/indi/venv/bin/python --version` on the live server to compare.

Step 1: Create the Linux user

Each installation runs as its own user, so its files, keys and data stay out of reach of the others. As root:

```
adduser --disabled-password --gecos "Indi installation 2" indi2
```

`--disabled-password` means nobody can log in as `indi2` with a password. You switch to it from root with `sudo -iu indi2`, which is how every later step starts.

Check it worked:

```
id indi2
ls -ld /home/indi2
```

The home folder should belong to `indi2`. Leave it at the default permissions (`drwxr-x---` on current Ubuntu), so the `indi` user can't read into it and the other way round.

Step 2: Give the installation read access to GitHub

The repository is private, so the server needs a key GitHub recognizes. A deploy key opens one repository only, and GitHub allows each key on one repository only. So every installation gets its own key, which also lets you cut one installation off without affecting the others.

On the server, as the new user:

```
sudo -iu indi2
mkdir -p ~/.ssh && chmod 700 ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/indi_github -N "" -C "indi2@$(hostname)"
cat >> ~/.ssh/config <<'EOF'
Host github-indi
  HostName github.com
  User git
  IdentityFile ~/.ssh/indi_github
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
cat ~/.ssh/indi_github.pub
```

The last command prints one line starting `ssh-ed25519`. Copy all of it.

On GitHub, signed in as indiartificial:

1. Open `github.com/indiartificial/indi-harness`, then **Settings → Deploy keys → Add deploy key**.
2. Title: something that names the installation, such as `indi2 on hetzner`.
3. Key: paste the line.
4. Leave **Allow write access** unchecked. This installation should pull code, never push it.
5. Click **Add key**.

Back on the server, test it:

```
ssh -T git@github-indi
```

The first time, answer `yes` to the host-key question. Success looks like: `Hi indiartificial/indi-harness! You've successfully authenticated, but GitHub does not provide shell access.` The exit code is 1 even on success; only the message matters.

Step 3: Clone the code and install Python packages

Still as `indi2`:

```
git clone git@github-indi:indiartificial/indi-harness.git ~/harness
cd ~/harness
git log --oneline -1
```

The commit shown should match the live install's latest pushed commit. Check that on the live side with `sudo -iu indi git -C ~/harness log --oneline -1`.

Then create the virtual environment and install the pinned packages:

```
python3 -m venv ~/venv
~/venv/bin/pip install --upgrade pip
~/venv/bin/pip install -r ~/harness/requirements.txt
~/venv/bin/pip check
```

`pip check` should print `No broken requirements found.`

Finally, run the test suite. It needs no accounts and sends nothing:

```
cd ~/harness && ~/venv/bin/python -m pytest -q
```

Every test should pass. A failure here means the code or packages differ from the live install, and it is worth fixing before going further.

If `pytest` is not found, it is not in `requirements.txt`. Install it with `~/venv/bin/pip install pytest`.

Step 4: Gmail account and Google Cloud

The installation needs its own Gmail mailbox. Never point two installations at one mailbox; they would answer the same mail twice.

4a. The Gmail account

Create a new Google account (or use a spare one) for the installation. Sign in once in a browser and finish any security prompts Google shows, so the account is fully active. Turn on 2-Step Verification. It protects the mailbox and stops Google from flagging the account later.

4b. Let the account use Indi's OAuth client

The simplest route reuses the Google Cloud project and OAuth client the live Indi already uses:

1. Go to console.cloud.google.com and select the project that holds Indi's OAuth client.
2. Open **APIs & Services → OAuth consent screen** (on newer consoles, **Google Auth Platform → Audience**).
3. While the app's publishing status is **Testing**, add the new Gmail address under **Test users** and save. Only listed test users can sign in.
4. If the app has been published, you can skip the test-user step.

Two things to know about Testing mode:

- Google caps it at 100 test users. That is plenty for your own installations, but not for customers.
- For an app in Testing, Google expires sign-ins that use Gmail scopes after about 7 days, and the installation then has to sign in again (Step 7). The live install is under the same rule, so whatever you do for it applies here too.

4c. Copy the OAuth client file

The client file, `credentials.json`, identifies the app, not the mailbox, so the same file serves every installation. As root:

```
cp /home/indi/harness/credentials.json /home/indi2/harness/
chown indi2:indi2 /home/indi2/harness/credentials.json
chmod 600 /home/indi2/harness/credentials.json
```

Do **not** copy `token.json`. That file is the live Indi's own signed-in access to its mailbox and Drive. A copy would make the new installation read and send as the live Indi.

For a customer's installation, they would use their own Google Cloud project and their own `credentials.json`. Then their usage and quotas stay separate from yours, and their sign-in screen shows their app.

Step 5: Anthropic API key

Give each installation its own key, so you can see each one's spending and revoke one without stopping the others.

1. At console.anthropic.com, open **API Keys** and create a key named for the installation (for example `indi2`).
2. Optional but sensible: give the installation its own **Workspace** with a monthly spend limit, and create the key inside it.
3. Copy the key; the console shows it only once.

On the server, as `indi2`:

```
cd ~/harness
nano api_key.txt        # paste the key on one line, save with Ctrl+O, exit
with Ctrl+X
chmod 600 api_key.txt
```

Indi logs a warning if `api_key.txt` is readable by other users, which is what the `chmod` prevents. The file is already listed in `.gitignore` and is one of the files Indi's own code tools refuse to open.

Step 6: The settings file (.env)

Every behavior switch lives in `~/harness/.env`. Start from the template in the repository, not from the live file, which carries the live install's addresses, IDs and alert topics.

```
cd ~/harness
cp .env.example .env
chmod 600 .env
nano .env
```

The template lists every setting with a comment. Change at least these:

Setting	Set it to	Why
`INDI_STAGING`	`false`	`true` turns on the virtual clock and the send allowlist meant for testing
`INDI_SEND_AUTOMATICALLY`	`false` for the first days	Replies wait in Drafts until you are satisfied, then switch to `true`
`INDI_APPROVE_NEW_CORRESPONDENTS`	`true`	New correspondents need operator approval before terms go out
`INDI_TIMEZONE`	The owner's zone, such as `America/New_York`	Dates and times in replies and the ledger
`INDI_ALLOWED_SENDERS`, `INDI_TRUSTED_SENDERS`	The owner's address, plus anyone else intended	Who Indi answers, and who gets owner-level treatment
`INDI_GIT_AUTO_PUSH`	`false`	This installation's GitHub key is read-only (Step 2), so pushing its own code changes would fail and raise an alert

Setting	Set it to	Why
`INDI_ENABLE_SELF_CODE`, `INDI_ENABLE_CLAUDE_CODE`, `INDI_ENABLE_LOCAL_EXEC`	`false`	Off until the installation has run cleanly for a while; these let mail change code or run commands
`INDI_SELF_CODE_SENDERS`, `INDI_CLAUDE_CODE_SENDERS`, `INDI_LOCAL_TOOLS_SENDERS`	Empty, or the owner only	Who may use those powers once switched on
`INDI_ALERT_NTFY_URL`	A new topic, or empty	Never the live topic, or the two installations' alerts mix
`INDI_HEALTHCHECK_URL`, `INDI_HEALTHCHECK_HEARTBEAT_URL`	New checks, or empty	Same reason
`INDI_CONSULT_PER_OWNER_PER_DAY`	`20`, as on the live install	Daily cap on owner consultations

These newer settings are fine at their defaults, which match the live install: `INDI_STANDING_NOTES=true`, `INDI_MEMORY_MODEL` (Sonnet), `INDI_JOURNAL_DIGEST=true` with `INDI_JOURNAL_DIGEST_DAYS=7`, `INDI_CALIBRATION=true`, and `INDI_NEWSLETTER_WEB_SEARCH=true` with `INDI_NEWSLETTER_MAX_SEARCHES=5`. Each installation keeps its own standing notes, linked addresses, claim record and memory-deletion queue in its own `data/` folder; none of them is copied from the live install.

If the template has a spreadsheet ID, Drive folder ID or similar storage setting, the new installation needs its own. Leave it empty, or create a new sheet or folder in the new Gmail account's Drive. Never reuse the live install's IDs: both installations would then write into the same records.

To see which settings the live install has that the new one does not, as root:

```
diff <(grep -o '^[A-Z_]*=' /home/indi/harness/.env | sort) \
     <(grep -o '^[A-Z_]*=' /home/indi2/harness/.env | sort)
```

This compares setting names only, never values. Lines marked `<` exist only on the live install; decide for each whether the new one needs it.

Start `data/` with nothing in it except the terms. Indi creates people, ledger, approvals and terms-acceptance files as it goes, and a fresh installation must not inherit the live one's correspondents. It does need the Terms of Service and user guidelines it sends to newcomers, which live in `data/terms/`. Check whether the clone brought them:

```
ls ~/harness/data/terms/
```

If the folder is missing or empty, copy just that folder from the live install, as root:

```
mkdir -p /home/indi2/harness/data
cp -r /home/indi/harness/data/terms /home/indi2/harness/data/
chown -R indi2:indi2 /home/indi2/harness/data
```

Step 7: Sign the installation in to Gmail

This step creates `token.json`, the installation's signed-in access to its own Gmail and Drive. Google's sign-in page has to send its answer back to a program on the server. An SSH tunnel from your PC makes the server's port reachable from your browser as if it were local, the same way the test accounts were signed in.

1. Find the sign-in command and port. As `indi2`:

```
cd ~/harness
grep -n -i "token.json\|sign in\|authori" README.md README-hetzner.md | head
-20
grep -rn "run_local_server" *.py
```

The first command finds the README's sign-in instructions. The second shows the port the code listens on (`port=...`). The steps below assume port 8090, as the test driver uses; substitute what you find.

2. Open the tunnel. In Windows PowerShell, log in to the server as root (the `indi2` user has no password), forwarding the port:

```
ssh -L 8090:127.0.0.1:8090 root@77.42.46.44
```

Leave this window open for the whole sign-in.

3. Run the sign-in as the new user. In that same window:

```
sudo -iu indi2
cd ~/harness
# the command from the README, for example:
~/venv/bin/python <sign-in script or command from step 1>
```

It prints a long `https://accounts.google.com/...` address and waits.

4. Sign in. Copy the address into a browser on your PC. Sign in as **the new installation's Gmail account**, not your own and not Indi's. Google warns that the app is unverified while it is in Testing: click **Continue**. Tick every permission box and continue. The browser lands on a page saying the flow is complete, and the server prints that it saved the token.

5. Lock it down and check it:

```
chmod 600 ~/harness/token.json
ls -l ~/harness/token.json
```

If the browser shows "connection refused" at the end, the tunnel is closed or uses a different port from the program. If Google says "access blocked", the account is missing from the test users (Step 4b).

Step 8: First test run

Run Indi by hand before any timer runs it, so problems show up on your screen rather than in a log.

1. A dry run. It reads the mailbox and decides what it would do, but sends, labels and records nothing. As `indi2`:

```
cd ~/harness && ~/venv/bin/python check_and_reply.py --dry-run
```

It should finish without a traceback. A fresh mailbox normally has nothing to do. Errors at this stage are usually a missing setting, the sign-in (Step 7) or the API key (Step 5).

2. A real message. From the owner's address, send the new installation's Gmail a short email with a body. Empty messages are skipped. Then run once for real:

```
cd ~/harness && ~/venv/bin/python check_and_reply.py
```

What to expect, in order:

1. With `INDI_APPROVE_NEW_CORRESPONDENTS=true`, an approval request goes to the operator. Reply **APPROVE** to it.
2. The next run sends the Terms of Service to the sender. Reply **I AGREE** from the owner's address.
3. The run after that sends the acceptance confirmation and the guidelines, and from then on Indi answers normally. With `INDI_SEND_AUTOMATICALLY=false`, replies appear in the new account's **Drafts** rather than going out.

Run the command again after each reply, rather than waiting for a timer. Note that the operator, for now, is always sjhalasz@gmail.com (see "What is still tied to the original operator").

Step 9: systemd services and timers

Two timers keep an installation running. The main one checks mail on a schedule; the heartbeat one does the periodic housekeeping. The unit files in `deploy/` are written for user `indi` and `/home/indi`, so the new installation gets renamed copies with its own user and paths.

1. See what is there. As root:

```
ls /home/indi2/harness/deploy/
```

You should see at least `indi.service`, `indi.timer`, `indi-heartbeat.service` and `indi-heartbeat.timer`.

2. Make the copies. As root:

```
N=indi2
cd /home/$N/harness/deploy
for f in indi.service indi.timer indi-heartbeat.service indi-heartbeat.timer;
do
  sed -e "s#/home/indi/#/home/$N/#g" \
      -e "s/^User=indi\$/User=$N/" \
      -e "s/^Group=indi\$/Group=$N/" \
      -e "s/Unit=indi\(-heartbeat\)\?\.service/Unit=$N\1.service/" \
      -e "/^OnFailure=/d" \
      "$f" > /etc/systemd/system/"${f/indi/$N}"
done
```

The last `sed` line drops any failure-alert hook. Those hooks run `indi-alert@`, which belongs to the live install and would report the new installation's failures as the live one's. Step 10 covers alerts properly.

3. Check nothing still points at the live install:

```
grep -n "indi\b\|/home/indi/\|User=\|Unit=\|ExecStart\|ReadWritePaths"
/etc/systemd/system/indi2*
```

Every path should say `/home/indi2/`, every `User=` and `Group=` should say `indi2`, and each timer's `Unit=` should name an `indi2` service. Fix any stray line by hand with `nano`.

4. Start it:

```
systemctl daemon-reload
systemctl start indi2.service          # one run now, as the service would do
it
systemctl status indi2.service --no-pager
journalctl -u indi2.service -n 50 --no-pager
systemctl enable --now indi2.timer indi2-heartbeat.timer
systemctl list-timers 'indi*'
```

The `start` of the service is the important check. It runs with the hardening settings in the unit (read-only home, writable `data/` only), which the manual run in Step 8 did not have. If it fails, the journal says which file it could not write; add that path to `ReadWritePaths=` in `/etc/systemd/system/indi2.service`, the same way it is listed for the live install.

`list-timers` should show both the `indi` and the `indi2` timers, each with its own next run time.

Pause and resume the new installation without touching the live one:

```
systemctl stop indi2.timer indi2-heartbeat.timer
systemctl start indi2.timer indi2-heartbeat.timer
```

Step 10 (optional): Alerts and the log viewer

The installation runs without either. Add them once it has run cleanly for a day or two.

Failure alerts

The live install's `indi-alert@.service` sends a phone notification when one of its units fails. The new installation gets its own copy, pointed at its own user, folder and ntfy topic. As root:

```
N=indi2
sed -e "s#/home/indi/#/home/$N/#g" -e "s/^User=indi\$/User=$N/" -e
"s/^Group=indi\$/Group=$N/" \
    /home/$N/harness/deploy/indi-alert@.service > /etc/systemd/system/$N-
alert@.service
for u in $N $N-heartbeat; do
  mkdir -p /etc/systemd/system/$u.service.d
  printf '[Unit]\nOnFailure=%s-alert@%%n.service\n' $N >
/etc/systemd/system/$u.service.d/alert.conf
done
systemctl daemon-reload
```

Alerts go to the topic in the new installation's `INDI_ALERT_NTFY_URL` (Step 6). Subscribe to that topic in the ntfy app on your phone.

To test it, briefly break the service and run it once:

```
mv /home/indi2/harness/api_key.txt /home/indi2/harness/api_key.txt.off
systemctl start indi2.service; sleep 5
mv /home/indi2/harness/api_key.txt.off /home/indi2/harness/api_key.txt
```

A notification should arrive within a minute.

Log viewer

The live viewer listens on port 8765. Give the new one a different port, such as 8766. As root:

```
N=indi2
sed -e "s#/home/indi/#/home/$N/#g" -e "s/^User=indi\$/User=$N/" -e
"s/^Group=indi\$/Group=$N/" \
    -e "s/--port 8765/--port 8766/" -e "/^OnFailure=/d" \
    /home/$N/harness/deploy/indi-logview.service > /etc/systemd/system/$N-
logview.service
systemctl daemon-reload && systemctl enable --now $N-logview
```

To view it from your PC, open a tunnel in PowerShell (`ssh -L 8766:127.0.0.1:8766 root@77.42.46.44`) and browse to `http://localhost:8766` .

To view it on the iPhone over Tailscale, serve it on its own HTTPS port alongside the live one, such as `tailscale serve --bg --https=8443 8766` . Then add an `INDI_LOGVIEW_HOSTS` drop-in for `indi2-logview` , the same way it was done for the live viewer. The address is the live one with `:8443` added.

Keeping it updated

New code reaches an installation only when you pull it; nothing updates on its own.

The usual flow:

1. A change lands on the live install (a patch you apply, or one of Indi's own approved code changes) and is committed there.
2. Push it from the live install: `sudo -iu indi git -C ~/harness push` .
3. Pull it on the other installation, then restart nothing. The timer picks up the new code on its next run:

```
sudo -iu indi2 bash -c 'cd ~/harness && git pull --ff-only && git log --
oneline -1'
```

If `requirements.txt` changed in the pull (`git diff --stat HEAD@{1} HEAD` shows it), also run:

```
sudo -iu indi2 ~/venv/bin/pip install -r ~/harness/requirements.txt
```

If anything in `deploy/` changed, redo the copy in Step 9 (and Step 10, if used), then `systemctl daemon-reload` .

If `.env.example` changed, compare it with `.env` for new settings: `diff <(grep -o '^[A-Z_]*=' .env.example | sort) <(grep -o '^[A-Z_]*=' .env | sort)` . A new setting missing from `.env` takes its default.

If `git pull --ff-only` refuses, someone changed files in this installation's copy. See what changed with `git status` and `git diff` . Never resolve that with `git reset --hard` without looking first: it would throw away anything Indi's self-code feature committed locally.

The staging copy (`~/harness-staging`) keeps pulling from the live install's folder as before. It does not need GitHub.

What is still tied to the original operator

The code still assumes Steve is the operator of every installation. That's fine for a second installation of your own. It has to change before anyone else runs one.

What	Where	Effect on a second installation	Fix
Operator address hardcoded to sjhalasz@gmail.com	`terms_gate.OPERATOR_ADDRESS`	Approval requests and emergency forwards go to Steve, and APPROVE/DENY replies are expected from Steve	An `INDI_OPERATOR_ADDRESS` setting in `.env`
Standing notes and operator-only tools follow that address	`standing_notes.py` , `calibration.py`	Proposed standing notes are emailed to Steve; only Steve can add notes directly or list and settle Indi's claims	Comes with the same setting
"Operated by Steven J. Halasz" in the covering note	`terms_gate.COVERING_NOTE`	Newcomers are told Steve runs this installation	An `INDI_OPERATOR_NAME` setting
The Terms of Service and user guidelines name Steve	`data/terms/` , `data/guidelines/`	Same as above, in the legal text	Terms written as templates, filled from the operator settings
Unit files assume user `indi` and `/home/indi`	`deploy/*.service` , `*.timer`	Needs the `sed` copies in Step 9	Generate the units from a template during installation
No license, serial number or payment check	(planned)	None today; any copy runs freely	The installer and license server in the separate licensing plan

To list every remaining spot, as `indi2`:

```
cd ~/harness && grep -rn "sjhalasz\|Halasz\|/home/indi\b" --include=*.py --include=*.txt --include=*.service --include=*.timer . | grep -v "^./tests/"
```

The first three rows are one small patch, and the natural first piece of the installer: it would ask for the operator name, email and phone, verify them, generate the serial number, and write `.env` and the unit files itself. With that in place, Steps 6 and 9 shrink to answering the installer's questions.

Troubleshooting

Symptom	Likely cause	What to do
`Permission denied (publickey)` on `git clone` or `ssh -T git@github-indi`	Deploy key not added, added to another repository, or `~/.ssh/config` missing	Recheck Step 2; `ssh -vT git@github-indi` shows which key was offered
`ERROR: Key is already in use` when adding the deploy key	That public key is already on another repository	Generate a new key for this installation (Step 2)
`pip install` fails building a package	Python version differs from the live server	Compare `python3 --version` on both; install the matching version or rebuild `requirements.txt` there
Google says "Access blocked" or "has not completed the Google verification process"	The Gmail address is not a test user	Add it under Test users (Step 4b) and sign in again
Browser shows "connection refused" at the end of sign-in	Tunnel closed, or port differs from the program's	Keep the PowerShell tunnel open; match the `-L` port to the program's

Symptom	Likely cause	What to do
`invalid_grant` or `Token has been expired or revoked` after about a week	Testing-mode sign-ins expire	Repeat Step 7
A message gets no reply and no approval request	Empty body, a sender not allowed by `.env`, or the installation is waiting for APPROVE or I AGREE	Run `check_and_reply.py` by hand and read its output; check `INDI_ALLOWED_SENDERS`
Replies sit in Drafts	`INDI_SEND_AUTOMATICALLY=false`	Intended at first; set it to `true` when satisfied
Manual run works, timer run fails	The service's hardening blocks a path	`journalctl -u indi2.service -n 50`; add the path to `ReadWritePaths=` (Step 9)
Both installations answer the same message	They share a mailbox or `token.json`	Each installation must have its own Gmail account and its own sign-in
Alerts or approvals arrive mixed with the live Indi's	Shared ntfy topic, or the hardcoded operator address	Give the installation its own topic (Step 6); the operator address needs the patch described above

For anything else, the first two places to look are the output of a manual run (Step 8) and `journalctl -u indi2.service`.